JAGAT GURU NANAK DEV PUNJAB STATE OPEN UNIVERSITY, PATIALA

DEPARTMENT OF COMPUTER SCIENCE & APPLICATIONS

M.Sc. (Computer Science) — Semester-IV

MAJOR PROJECT VIVA VOCE SLIDES & SPEECH SCRIPT

Course Code: MSCS-4-03P | Major Project (M.Sc. Computer Science, Semester-IV)

Project Title:

Design and Implementation of a High-Throughput, Offline-First, Multi-Tenant SaaS Enterprise POS Architecture with Integrated AI Copilot and Computer Vision OCR

University: Jagat Guru Nanak Dev Punjab State Open University, Patiala (JGND PSOU)
Target Duration: 10–15 Minutes Presentation + 7 Minutes Viva Voce Discussion
Total Slides: 15 Slides

Slide 1: Title & Candidate Particulars

Header: Jagat Guru Nanak Dev Punjab State Open University, Patiala

Degree: Master of Science in Computer Science — Semester IV (Final Semester)

Title: Design and Implementation of a High-Throughput, Offline-First, Multi-Tenant SaaS Enterprise POS Architecture with Integrated AI Copilot and Computer Vision OCR

Candidate: `Sumit Singh` | Roll No: `PSOU/MSCS/2026/042`

Supervisor: `Dr. Harpreet Singh`

Spoken Speech Script (00:00 - 01:00):
"Respected External Examiner, Internal Supervisor, and Faculty Members. A very good morning. Today, I am proud to present my M.Sc. Computer Science final major project dissertation titled 'Design

and Implementation of a High-Throughput, Offline-First, Multi-Tenant SaaS Enterprise POS Architecture with Integrated AI Copilot and Computer Vision OCR'—codenamed SSR ONE AI. This project addresses the fundamental operational vulnerabilities in modern commercial hospitality systems: network-induced counter latency, offline failure during internet blackouts, and cross-tenant data security risks. Over the next 12 minutes, I will demonstrate how our distributed architecture solves these challenges."

Slide 2: Motivation & Problem Statement

Visual Problem Taxonomy:

1. The Latency Wall: Synchronous cloud HTTP round-trips cause 150ms–800ms delays per ticket, generating long cashier queues.

2. The Network Blackout Cliff: Commercial kitchens and mall basements are RF dead-zones; internet drops paralyze counters.

3. Cross-Tenant Vulnerability: In shared SaaS databases, relying on manual application-level filters creates severe data leak risks.

Spoken Speech Script (01:00 - 02:15):
"In commercial hospitality and retail, speed is an existential operational requirement. Traditional cloud SaaS architectures force cashiers to wait for remote round-trip database commits. During high-pressure dinner rushes, network jitter or localized Wi-Fi drops freeze the cash register, causing double billing and angry patrons. Furthermore, in multi-tenant SaaS environments, relying on software developers to remember 'WHERE tenant_id = X' on every SQL query is an unacceptable security hazard. Our motivation was to build an enterprise platform that guarantees sub-millisecond local billing, 100% offline continuity, and kernel-level multi-tenant security."

Slide 3: Project Objectives & Research Scope

Specific Research Objectives:

Obj 1: Sub-millisecond ($< 0.5\text{ms}$) local-first billing via Zustand and Dexie.js (IndexedDB).

Obj 2: High-throughput asynchronous backend via Python 3.11, FastAPI (ASGI), and PostgreSQL 16.

Obj 3: Kernel-enforced multi-tenancy via PostgreSQL Row-Level Security (RLS).

Obj 4: Real-time kitchen dispatch ($< 25\text{ms}$) via WebSockets and asynchronous ESC/POS thermal printing.

Obj 5: Multimodal AI Copilot (SSE streaming) and localized Tesseract OCR for automated supplier invoice scanning.

Spoken Speech Script (02:15 - 03:30):
"To solve these problems, we formulated five precise technical objectives: First, develop an offline-first transactional engine inside the browser using IndexedDB. Second, construct a high-throughput asynchronous backend on FastAPI and async PostgreSQL. Third, mathematically eliminate cross-tenant data leaks using PostgreSQL Row-Level Security. Fourth, deliver sub-second kitchen ticket dispatch over full-duplex WebSockets. And fifth, embed an intelligent multi-agent AI Copilot alongside localized computer vision OCR to automate supplier invoice digestion without recurring cloud API costs."

Slide 4: Overall System Architecture Blueprint

Visual Architectural Map:

Monorepo Structure: Turborepo + pnpm workspaces (`apps/` , `packages/` , `services/`).

Client Layer: React 19, Zustand, Dexie.js, TanStack Router.

Gateway: Nginx SSL termination, CORS regex filter, Gzip.

Backend: FastAPI ASGI event loop, Celery workers, Redis.

Persistence: PostgreSQL 16 (Neon) with RLS + PgBouncer pooler.

Spoken Speech Script (03:30 - 04:45):
"Here is the overarching architectural blueprint of SSR ONE AI. Constructed as an enterprise monorepo using pnpm and Turborepo, the platform cleanly decouples frontend web applications from shared type packages and backend microservices. The client layer runs React 19 with local IndexedDB storage. Requests pass through an Nginx reverse proxy to our FastAPI ASGI backend. Heavy tasks like PDF generation and OCR scanning are offloaded to asynchronous Celery workers running on Redis queues, while PostgreSQL 16 provides transactional ACID persistence with Row-Level Security."

Slide 5: The Zero-Wait Offline Billing Mechanism

Visual Dataflow:

Cashier Tap ➡ Zustand In-Memory Mutation ($< 0.05\text{ms}$) ➡ Dexie.js Write (0.38ms) ➡ UI Form Cleared Immediately.

Decoupled Background Worker: `useZeroWaitOrderSync.ts` asynchronously drains orders to server.

Idempotency: RFC4122 UUIDv4 tokens eliminate duplicate billing during network retries.

Spoken Speech Script (04:45 - 06:00):
"Let us examine our primary innovation: the Zero-Wait Offline Billing Engine. When a cashier clicks 'Send to Kitchen', the order is written directly into the browser's native IndexedDB database via Dexie.js in just 0.38 milliseconds. The UI clears instantly so the cashier can serve the next customer. Notice what does NOT happen: we do not await the remote cloud server. A decoupled background

worker monitors network state and drains orders asynchronously using unique UUIDv4 idempotency keys. Even if the restaurant's router is physically unplugged for 30 minutes, billing operations continue at 100% capacity."

Slide 6: Multi-Tenant Database Security via PostgreSQL RLS

Visual DDL Snippet & Flowchart:

`ALTER TABLE orders ENABLE ROW LEVEL SECURITY;`

`CREATE POLICY tenant_isolation_policy ON orders USING (tenant_id = current_setting('app.current_tenant_id')::bigint);`

Session Binding: `SET LOCAL app.current_tenant_id = '...'` injected on every connection checkout.

Spoken Speech Script (06:00 - 07:15):
*"For cybersecurity, we rejected standard application-layer filters. Instead, we enforce security at the PostgreSQL database kernel using Row-Level Security. Every single table inherits from TenantBaseModel. When a request arrives, our FastAPI dependency extracts the tenant ID from the cryptographically signed JWT and executes SET LOCAL app.current_tenant_id. From that exact microsecond, PostgreSQL's query engine physically blocks any SQL statement from accessing rows belonging to another tenant. Even if an engineer wrote 'SELECT * FROM orders', the database automatically restricts the result set strictly to that tenant."*

Slide 7: High-Throughput Asynchronous Backend (FastAPI & ASGI)

Visual Benchmarking Comparison:

WSGI (Thread-per-request) vs. ASGI (Coroutine Event Loop).

Non-blocking `asyncpg` connection pool with PgBouncer.

Concurrency Handling: Atomic sequence token generation (`with_for_update()`).

Spoken Speech Script (07:15 - 08:30):
"Our backend is built on FastAPI and Uvicorn. Unlike traditional synchronous WSGI frameworks like Django or Flask that tie up an entire OS thread per request, ASGI uses an asynchronous coroutine event loop. When a query is waiting on database disk I/O, the event loop immediately serves other incoming connections. Furthermore, to solve the race condition where multiple cashiers submit orders at the exact same millisecond, we implemented an atomic sequence counter using PostgreSQL's 'SELECT ... WITH FOR UPDATE' row lock, guaranteeing collision-free daily tokens like #042 without duplicate numbers."

Slide 8: Real-Time Kitchen Display System (KDS) & WebSockets

Visual Screen Mockup & Diagram:

Full-duplex WebSocket connections (`KDSConnectionManager`).

Ping/Pong heartbeats with auto-reconnect backoff (3s).

Sub-second ticket broadcast SLA: $< 25\text{ms}$.

Decoupled asynchronous ESC/POS thermal printing queue worker.

Spoken Speech Script (08:30 - 09:30):

"In a fast-paced commercial kitchen, paper tickets get lost or soaked in grease. We developed a digital Kitchen Display System operating over full-duplex WebSockets. The moment an order commits to the database, our internal event bus broadcasts the ticket to kitchen display terminals in under 25 milliseconds. The screen chimes and color-codes tickets by cooking station (Tandoor, Grill, Curry). Simultaneously, our asynchronous print queue dispatches raw ESC/POS bytes over the local network to thermal receipt printers without stalling the cashier's screen."

Slide 9: Multimodal AI Copilot & Computer Vision OCR

Visual Architecture:

Multi-Agent Personas: CEO, Kitchen, CRM, Finance.

Streaming: Server-Sent Events (`text/event-stream`).

Localized Tesseract OCR: Grayscale ➡ Otsu Binarization ➡ Pytesseract Tabular Invoice Extraction.

Spoken Speech Script (09:30 - 10:45):

"To modernize administrative operations, we integrated two advanced intelligence engines: First, a multi-agent AI Copilot featuring specialized personas for the CEO, Chef, and Accountant. Responses stream token-by-token over Server-Sent Events with full audit logging in PostgreSQL. Second, a localized computer vision pipeline powered by Tesseract OCR and OpenCV. When vegetable or grocery deliveries arrive, staff photograph the paper bill. The system removes visual noise, binarizes the text, extracts line items and GST tax totals, and auto-populates draft purchase orders without paying recurring cloud vision API fees."

Slide 10: Empirical Performance Benchmarks

Visual Results Table:

P50 Latency: Cloud-Only = 142.3 ms | SSR ONE AI = **0.38 ms (99.7% Faster)**.

P99 Latency: Cloud-Only = 1,420 ms | SSR ONE AI = **0.89 ms**.

Offline Resilience: Cloud-Only = 0% Success | SSR ONE AI = **100% Success**.

UI Frame Rate: Cloud-Only = 18–35 FPS | SSR ONE AI = **Locked 120 FPS**.

Spoken Speech Script (10:45 - 11:45):
"Let us examine the empirical benchmark data gathered from 1,000 synthetic retail checkout transactions. As shown on the slide, our offline-first architecture achieved a P50 median local commit latency of just 0.38 milliseconds compared to 142.3 milliseconds for traditional cloud POS—a 99.7% reduction in operational latency. Under 2% network packet loss, traditional cloud checkouts suffered an 7.8% failure rate, whereas our system experienced 0% failure. During a simulated 30-minute complete router blackout, our terminal processed 150 orders seamlessly, syncing all records in 4.2 seconds upon network return."

Slide 11: System Demonstration & Live Screenshots

Visual Screenshots of Working Platform:

POS Billing Screen: Table layout, rapid dish search, instant cart updates.

KDS Screen: Live kitchen tickets updating in real-time.

AI Copilot Screen: Executive business insights streaming over SSE.

Admin Dashboard: Multi-tenant metrics and branch analytics.

Spoken Speech Script (11:45 - 12:45):
"These screenshots illustrate the production system in action. The top image shows the POS billing interface with zero-wait cart updates. The bottom-left shows the kitchen KDS terminal receiving real-time tickets over WebSockets. And the right image demonstrates our AI Copilot providing executive profit analysis and inventory alerts directly from live database metrics."

Slide 12: Security & Concurrency Verification

Visual Verification Checklist:

- ☑ **Zero Cross-Tenant Leakage**: 100% verified via automated penetration test scripts.
- ☑ **Zero Token Duplication**: 500 concurrent requests yielded exact sequential `#001` - `#500` numbers.
- ☑ **Brute-Force Lockout**: 5 failed login attempts trigger automated 15-minute account freeze.
- ☑ **Audit Trail Compliance**: All voids, refunds, and AI prompt queries recorded in `audit_logs`.

Slide 13: Conclusions & Contributions

Key Achievements:

Proved that local-first IndexedDB persistence eliminates user counter latency.

Established a bulletproof multi-tenant security architecture via PostgreSQL RLS.

Delivered an end-to-end commercial platform uniting POS, KDS, ERP, AI, and OCR.

Slide 14: Academic Publications & References

Research Paper Draft:

"A Resilient Offline-First Multi-Tenant Cloud Architecture for High-Concurrency Point-of-Sale Systems" (Submitted for conference publication).

Key Citations: Kleppmann et al. (2019), Brewer (2012), Curino et al. (2010), Smith (2007).

Slide 15: Acknowledgments & Floor Open for Viva Voce

Visual Text:

Thank You to the Respected Examiners and Faculty.

Ready for Viva Voce Examination & Technical Defense.